Communication Loss Is Not Failure: Consensus-Free Task Ownership for UAV Swarms Under Network Partition

Charalampos Nadiotis
ADDITESS Ltd, Nicosia, Cyprus
Frederick University, Nicosia, Cyprus
Team Teleforce, EUDIS Defence Hackathon 2026
charalamposnadiotis@gmail.com

***Abstract*—Radio interference, terrain and range routinely split small UAV swarms into groups that cannot hear one another, and a silent aircraft may be jammed, out of range, or destroyed. Coordination schemes that reassign work only once the swarm *agrees* that an aircraft has failed stop making progress exactly when the environment stops delivering agreement. We present a receiver-local coordination architecture in which every control transition of a UAV is justified only by evidence that this UAV has actually received. Task ownership is a replicated, lease-based claim with owner-scoped epochs. Conflicting claims created during a partition are permitted and are reconciled deterministically after reconnection, while quorum-certified failure declarations are kept off the recovery path. A composed set of extended finite-state machines exposes each UAV's contact, peer-availability, ownership, coordination and mission posture, and a versioned operator contract (CONTINUE/HOLD/ABORT) governs degradation. In paired deterministic simulations the distributed protocol and a centralized coordinator detect losses identically at every communication range, yet the coordinator fails 68 of 220 jammed missions while the distributed protocol fails none. With two aircraft, where a loss is undeclarable by construction, only the consensus-free mode completes. We quantify the price of availability (0.29 task-seconds of transient double ownership per mission, against 4.8 task-seconds of stranded work under central control) and show that a HOLD policy removes 64% of it for 59% more mission time. The same protocol coordinates sector search and human-confirmed engagement on eight ArduCopter software-in-the-loop vehicles, with an advisory QAOA planner that we benchmark against the exact optimum and random sampling. We report the limitations openly, including a $1.7\times$ timing error of the point-mass model measured against autopilot firmware.**

***Index Terms*—UAV swarms, distributed task allocation, network partition, failure detection, leases, extended finite-state machines, contested communications, human-in-the-loop, ArduPilot SITL, QAOA.**

I. Introduction

A SWARM of small UAVs is attractive for intelligence, surveillance and reconnaissance (ISR) because the mission can, in principle, degrade gracefully as individual aircraft are lost [1], [2]. In practice, that promise depends on communication, and contested environments deny it. As of September 2026, the European Union Aviation Safety Agency lists the Nicosia Flight Information Region (LCCC) among the regions affected by GNSS jamming and spoofing [3], [4], and a peer-reviewed flight experiment in that region recorded a multimode receiver losing positioning for 80% of the experiment time [5]. Those sources concern satellite navigation rather than inter-vehicle links, and we do not use them to calibrate anything. They show that an RF environment can remove the shared picture that coordination depends on, for an interval nobody can predict, while every aircraft involved is still airborne and still tasked.

When a UAV falls silent, its peers cannot tell whether it is jammed, behind terrain, briefly out of range, or destroyed. A design that reallocates work only after the swarm agrees that the aircraft is lost has made progress conditional on agreement, and agreement is exactly what a partition removes. Deterministic consensus is impossible in an asynchronous system with even one crash failure [6], a replicated service cannot be both consistent and available under partition [7], and any practical failure detector is unreliable [8]. The operational consequence is blunt: *a swarm that requires agreement in order to reallocate work stops working at the moment the environment stops delivering agreement.*

We therefore treat communication loss as a normal operating condition, not an exception, and organise the design around one rule:

$$\text{communication loss} \neq \text{UAV failure}.$$

Each UAV decides only from evidence it has received. The system permits *temporary disagreement*: during a partition, two groups may each legitimately own the same task. The guarantee is deliberately weaker than global consensus. After connectivity is stably restored, every replica converges on the same owner for every task, and the losing claimant stands down on its own. Work held by a silent owner is recovered by *lease expiry*, which requires no agreement. Quorum-certified failure declarations are still produced, but as belief for situational awareness rather than as a precondition for recovery.

The design began as a hand-drawn concept of operations (CONOPS) in which the operator provides mission intent and the UAVs autonomously and distributively allocate tasks, cooperate using local and selectively shared information, and

continue despite vehicle or communication failures; no UAV needs complete knowledge of the swarm and no single node is required for the mission to continue (Fig. 1; original notebook pages in Appendix A). This paper reports what that design became, what we measured, and where its evidence stops. Its contributions are:

1) a *receiver-local* coordination architecture with an explicit epistemic rule (every control guard is a function only of evidence delivered to that UAV and of its own clock), enforced structurally by a three-layer state separation and by architecture tests (Sections III–V);
2) a *consensus-free task-ownership protocol*: lease-based immutable claims with owner-scoped epochs, deterministic lowest-identifier reconciliation with exact-generation releases and absorbing completion, and quorum-certified, retractable failure declarations. We state its key properties, including a declaration-free recovery bound (Section IV);
3) a composed per-UAV *extended finite-state machine* (EFSM) kernel and a versioned operator mission contract whose degradation policy can only add restraint (Section IV-G);
4) a paired, deterministic evaluation over more than 2,500 simulated missions. Distributed and centralized control show identical loss awareness but opposite outcomes. We add a quorum-floor counterexample, measure the price of availability, and show how the gap depends on geometry and fault timing (Section VI);
5) a ground-node *live mission* in which the same protocol coordinates sector search and human-confirmed engagement on eight ArduCopter software-in-the-loop (SITL) vehicles. It includes an advisory QAOA assignment planner benchmarked honestly against the exact optimum and against random sampling, and a measurement of point-mass model error against autopilot firmware (Section VII).

II. Related Work

Multi-robot task allocation. Allocation taxonomies [9] and auction/consensus methods such as CBBA [10] reach conflict-free assignments when bids eventually propagate. Recent communication-aware allocators combine topology construction with robust auction-consensus and report resilience to packet loss, delay and topology switching in simulation [11]. Communication-aware allocation is therefore not new. Our target is different: what a UAV may *execute* while bids cannot propagate at all, and how replicas converge afterwards, without importing a globally coherent bid view.

Partition-tolerant replicated state. Surveys of UAV networking describe intermittent links and rapidly changing topologies as the norm in flying ad-hoc networks [12], [13]. Availability-first replication with eventual convergence is well established in distributed systems [14]–[16], built on epidemic dissemination [17]. Leases bound the validity of cached authority without synchronous revocation [18]. In swarm robotics, SwarmDAG lets disconnected groups continue recording progress

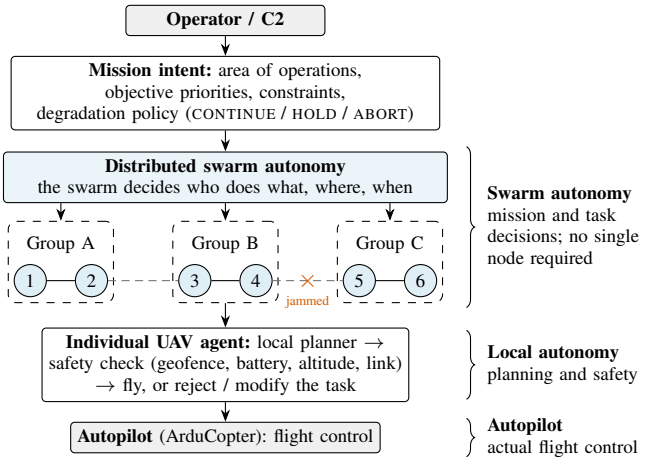


Fig. 1. Concept of operations, redrawn from the team's design notebook (Appendix A). The operator supplies intent, not a plan. Groups that lose contact (the jammed link) continue from local knowledge and synchronise when contact returns. Each UAV can reject a swarm task that fails its local safety check.

and later merge toward eventual consistency [19], and the CHARISMA project describes a distributed shared-memory service for heterogeneous unmanned vehicles [20]. Partition tolerance by itself is thus prior art. We claim no novelty for it. Our contribution is in the receiver-local semantics of ownership, evidence and execution authority, and in measuring what they buy.

Failure detection and consensus. Consensus-based reassignment, e.g., Paxos [21] or Raft [22], needs a majority that can communicate. Unreliable failure detectors [8] formalise why silence is ambiguous. We keep a quorum-certified declaration, but we deliberately decouple task recovery from it.

Formal discrete control. Supervisory control theory has been applied to robot swarms with up to 600 physical robots [23]; statechart-based mission execution separates UAS mission logic from flight control [24]; automata-based frameworks synthesise multi-agent plans from capability models [25]. Industrial interfaces also expose explicit, versioned task lifecycles with terminal states [26]. Following this line, we keep finite-state structure around discrete decisions and leave radio, geometry and optimisation numerical.

Search and coverage. Boustrophedon decomposition [27], [28], communication-constrained coverage [29], and distributed receding-horizon 3-D search with a varying number of agents [30] address *how* an area is searched. We use the simplest of these (sector strips with boustrophedon lanes) only to exercise *who* may search each sector. CARIC [31] is a candidate external benchmark for future comparisons.

Quantum optimisation. QAOA [32] and the quantum alternating operator ansatz with constraint-preserving XY mixers [33], [34] apply to QUBO-encoded assignment problems [35]. Practical advantage remains problem- and hardware-dependent [36], and simulated QAOA has been proposed for network subproblems [37]. We use QAOA only as an advisory planner and report it against classical and

random baselines.

III. System Model and Design Rule

Agents and tasks. A swarm has N UAVs with distinct, totally ordered identifiers $V = \{1, \ldots, N\}$. The mission is an immutable catalogue $\mathcal{J}$ of objectives (identifier and position) handed to every UAV at start. In the live mission (Section VII), new objectives are discovered from geolocated detections. Mutable task records exist only as an observer-facing projection and are never an input to control.

Communication. The world model maintains a time-varying undirected graph $G(t) = (V, E(t))$. In the simulator an edge exists if both endpoints are within a communication range R, the link is not explicitly blocked, and no active region jammer covers the link's midpoint. An optional free-space line-of-sight link model with seeded stochastic delivery also exists, but it is an analytical abstraction, not calibrated RF. Messages travel hop by hop over $E(t)$ only. Network conditions range from connected, through degraded and partitioned, to an isolated UAV; we treat all of them as normal operating modes.

Faults and clocks. UAVs fail by stopping (fail-stop): a failed UAV neither moves, transmits, relays nor renews. We do not model Byzantine behaviour or identity forgery. Each UAV measures ages on its own clock from the moment it first accepted a piece of evidence. Source timestamps are metadata and are never compared with the receiver's clock to make a decision. The simulator happens to drive every replica from one deterministic logical clock, but decision logic does not rely on that, and no decision orders events across UAVs by their clocks [38].

Definition 1 (Receiver-local knowledge and the epistemic rule): Let $K_i(t)$ be UAV i's own state together with every evidence item delivered to i up to time t, each stamped with its local receipt time. A control transition of i at time t is *admissible* only if its guard is a function of $K_i(t)$, t and i alone.

The rule excludes, in particular, graph reachability (a radio cannot report that a link is down; only silence accumulates), another UAV's live position or task, mission-wide failure projections, and anything computed by the evaluator. Table I lists the three kinds of state that coexist in one process and the direction in which information may flow between them. Architecture tests fail the build if, for example, the autonomy module imports the simulator or the dashboard imports authoritative world state.

Design goals. (G1) *Availability under partition:* every connected group keeps working on unowned objectives. (G2) *Convergence:* after connectivity is stably restored, every task has one owner at every replica. (G3) *No single required node* in the protocol, including a coordinator or a ground station on the coordination path (the live harness of Section VII still co-locates all agents on one ground node; see Section VIII). (G4) *Operator restraint:* the operator chooses how much risk of duplicated work the mission accepts. (G5) *Human authority over engagement* in the live mission. Fig. 2 shows how the components of each UAV's agent, as first sketched in the design notebook, map onto the three layers and onto the implementation.

TABLE I
Three kinds of state and what each must not do

Layer	Holds	Must not
World / physics	true positions and responsiveness, $G(t)$, radio and jammer models, observer task projection	supply remote state or graph truth to decisions; authorise ownership
Receiver-local control	own state, task catalogue, local utility, peer-state store, claim ledger, EFSM kernel, held contract	infer failure from silence alone; act without local ownership
Observer / evaluation	metrics, traces, dashboard, operator map, test assertions	feed its privileged view back into any decision

IV. Receiver-Local Coordination Protocol

A. Evidence and dissemination

Two kinds of message exist. *Heartbeats* are one-hop by design: receiving one means first-hand contact with its source. Everything else is *immutable protocol evidence*: task claims, releases, completions, failure votes, failure declarations and mission-contract versions. Immutable evidence is flooded deterministically over the current connected component (store-and-forward). Each item has a content-derived structural identity, e.g., (j, o, e) for a claim. Every receiver applies an item at most once, forwards it to neighbours in sorted order, and retains failed directed routes for retry when the topology changes. Finite membership and per-node seen-state make flooding terminate without a time-to-live. Forwarding metadata (logical origin, forwarder, receiver, hop count) lives in the transport envelope; a relay never becomes the author of what it carries. A duplicate delivery never refreshes the age of evidence it repeats.

B. Peer status and quorum-certified failure

Each UAV keeps a peer-state store that derives, for every peer, one of three statuses (HEARD, SILENT or DECLARED_FAILED) from delivered heartbeats and elapsed local time. There is no status meaning "the link is down". A UAV may create a *failure vote* for a peer only after hearing nothing for strictly longer than the heartbeat timeout. It may create a *declaration* only when its own mailbox holds enough fresh, matching votes, its own among them. The threshold is

$$q(N) \;=\; \max\Big(2,\; \big\lfloor (N-1)/2 \big\rfloor + 1\Big). \tag{1}$$

Proposition 1 (No unilateral declaration): For every N, $q(N) \geq 2$: no UAV can certify a peer's failure from its own evidence alone. For $N = 2$ no failure can ever be declared.

Proof: Immediate from (1); for $N = 2$ the only possible voter is the single survivor, and $1 < q(2) = 2$. ∎

Proposition 2 (Quorum intersection): For any suspect s and $N \geq 3$, any two voter sets that each reach $q(N)$ intersect. Two groups that exchange no votes therefore cannot both certify s from votes created within each group.

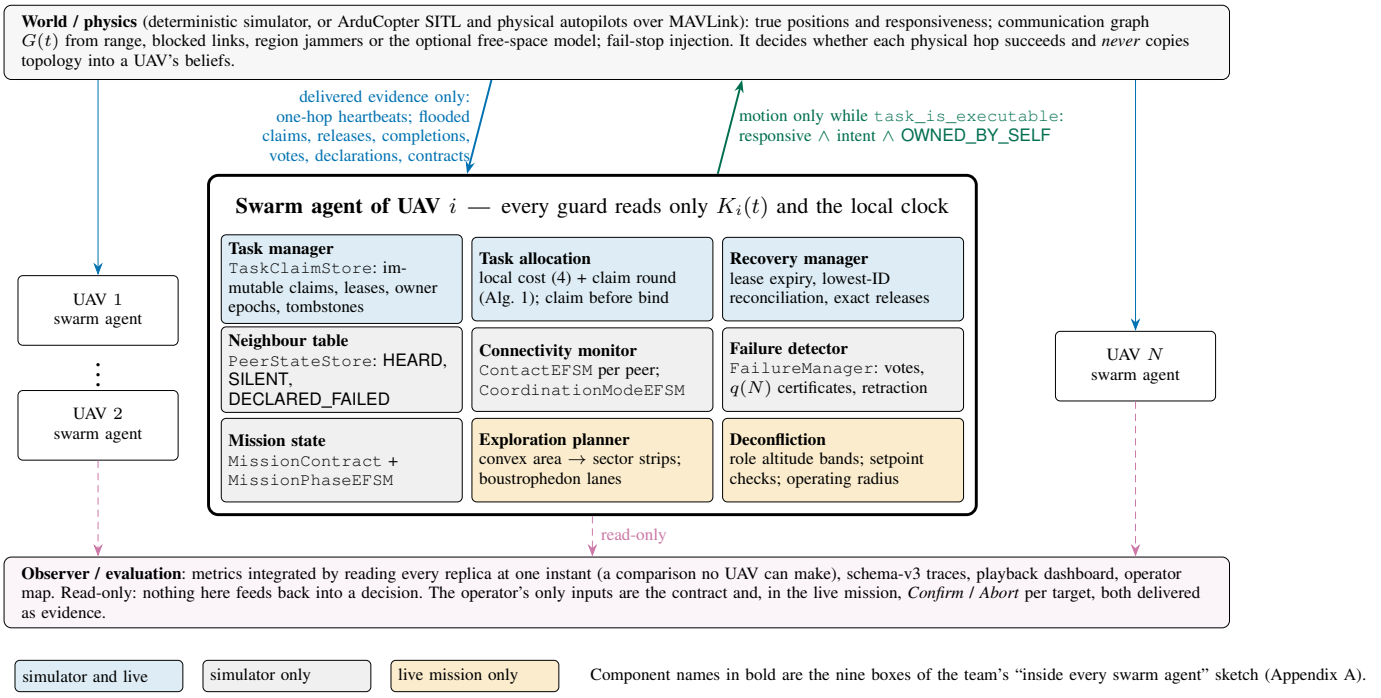


Fig. 2. Architecture. The world layer gates every physical hop but never supplies beliefs; each UAV's agent decides only from delivered evidence; the observer layer reads everything and writes nothing. Shading shows where each sketched component is implemented today.

Proof: Voters are drawn from $V \setminus \{s\}$, which has $N-1$ members. For $N \geq 3$, $q(N) = \lfloor (N-1)/2 \rfloor + 1 > (N-1)/2$, so two disjoint qualifying sets would need more than $N-1$ members. ∎

A certificate is *corroborated belief, not proof*: there is no positive remote-health signal, so a quorum can wrongly declare a healthy UAV that sits behind a partition. Declarations are therefore reversible: a newly delivered heartbeat from the declared peer outranks a second-hand certificate and retracts it. A declaration never stops a vehicle. A wrongly declared UAV keeps flying and keeps the task it does not know it lost; ownership reconciliation, not the detector, repairs that.

C. Lease-based task ownership

Each UAV keeps an independent claim ledger (`TaskClaimStore`). A claim $c = (j, o, e)$ states that owner o claims task j at owner-scoped epoch e; it is an immutable value. Receiver i records the time $\rho_i(c)$ at which it first accepted c and computes the age $a_i(c,t) = t - \rho_i(c)$. With a freshness threshold F and a lease duration L, $0 < F < L$, the claim is

$$\text{fresh if } a \leq F, \quad \text{stale if } F < a \leq L, \quad \text{expired if } a > L. \tag{2}$$

Stale is not free: a stale claim is still lease-valid, and a UAV may create a competing claim only when every incompatible claim it knows is expired or explicitly released. An owner renews by issuing the next epoch when its own copy reaches age F (default $F = L/2$), not on every tick. Epochs are compared only within one (j, o) stream: a higher epoch supersedes lower ones from the same owner, a delayed lower epoch is ignored, the same (j, o, e) with a different payload is rejected as contradictory, and a release tombstone prevents a delayed copy from resurrecting ownership. Epochs from different owners are never compared.

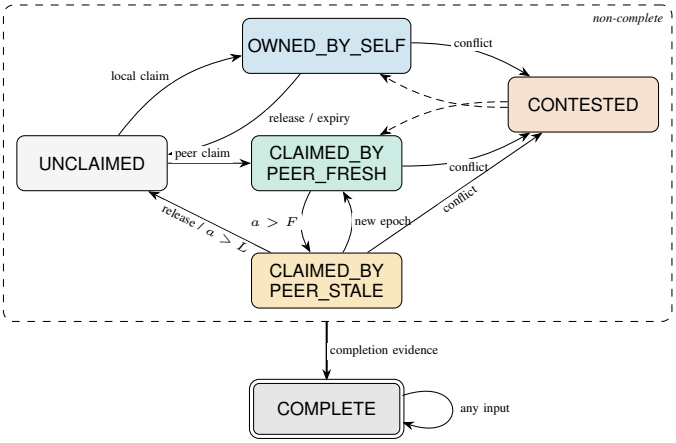


Fig. 3. Receiver-local task-ownership interpretation S_{ij} (one machine per task per UAV). "Conflict" means an incompatible lease-valid claim becomes visible; dashed arrows are reconciliation outcomes (self or peer; selecting a stale peer is also possible, edge omitted). COMPLETE is absorbing.

From its ledger, receiver i derives a six-state interpretation $S_{ij}(t)$ of task j (Fig. 3). The vocabulary is deliberately exact. UNCLAIMED means "i has no lease-valid claim evidence for j", not "nobody owns j". Two partitioned UAVs may both report OWNED_BY_SELF; this split-brain state is allowed and represented, not a bug.

D. Deterministic reconciliation and completion

When more than one owner's latest claim is lease-valid at receiver i, then $S_{ij} = \mathsf{CONTESTED}$. Freshness does not break the tie: a stale but lease-valid claim is a real contender. Conflict handling has three explicit phases. *Observe*: the receiver keeps every contender and reports CONTESTED. *Reconcile*: it selects the claim with the lowest owner identifier,

$$w_{ij}(t) \;=\; \min\{\, o \;:\; (j, o, \cdot) \in \mathcal{C}_{ij}(t) \,\}, \tag{3}$$

where $\mathcal{C}_{ij}(t)$ is the set of latest lease-valid, unreleased per-owner claims known to i. *Release*: if i itself lost, it stops acting as owner immediately and floods an exact-generation release that names the winning claim. Completion evidence can be created only by the current lease-valid owner and carries the claim it completes. Once accepted, it makes $S_{ij} =$ COMPLETE permanently, whatever arrives later.

Proposition 3 (Agreement and post-partition convergence): (a) Any two receivers with the same set of owners in $\mathcal{C}_{ij}$ select the same winner, independently of message order. (b) Suppose that from time t_h the responsive UAVs form a connected component of $G(t)$ that remains stable, that each claimant keeps renewing while it believes it owns j, and that no new conflicting claim is created. Then within the time needed to flood the renewals and the losers' releases, every replica in the component holds the single owner $w = \min$ of the claimants.

Proof sketch: (a) Equation (3) is a function of the owner set over a total order. (b) Flooding delivers every immutable item to every node of a stable connected component in at most its diameter of rounds. Each claimant renews at age $F < L$, so once renewals are delivered all replicas hold every claimant's latest claim as lease-valid and agree on w by (a). Each loser ℓ holds the same set, so it releases. Its release floods, and tombstones remove ℓ's generation everywhere. Thereafter ℓ sees j as CLAIMED_BY_PEER_* and cannot re-claim while w renews. ∎

Proposition 3 is argued, not machine-checked. It is exercised by deterministic integration tests, including split-brain reconciliation through the four-node chain $1{-}2{-}3{-}4$. Note that the lowest-identifier rule is a correctness mechanism that makes replicas agree, not a utility-optimal allocator.

E. Recovery without declaration

Proposition 4 (Declaration-free recovery bound): Let owner o of task j fail-stop at time t_f, and let receiver r accept o's last claim generation at time ρ_r. If r knows of no other valid claim on j, then j is claimable at r for every $t > \rho_r + L$, whether or not any failure declaration about o exists. If that generation was created at or before t_f and reached r after a dissemination delay of at most δ, the task is claimable at r after $t_f + \delta + L$.

Proof: A fail-stopped owner creates no new epoch. By (2) the copy at r expires once $a_r > L$, and no guard on the claim path reads peer status. The second statement follows from $\rho_r \le t_f + \delta$. ∎

A coordinator that reassigns only on a declaration has no such bound. The declaration requires $q(N)$ responsive voters, connected to one another and to the coordinator. If that never happens, as with $N = 2$ (Proposition 1) or a sparse or jammed graph, the task is held by a stopped aircraft indefinitely. Fig. 4 contrasts the two paths. Activation additionally needs an idle UAV, so the bound is on *availability*, not on completion.

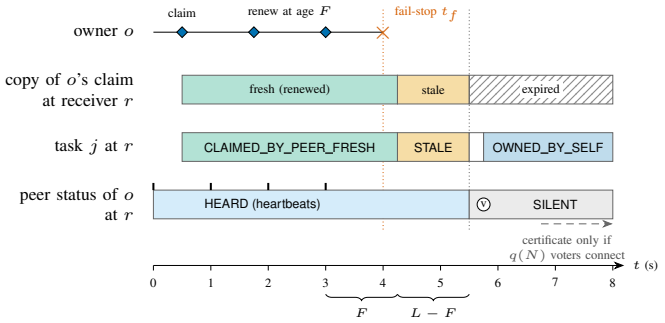


Fig. 4. Recovery from a fail-stop with the default $L = 2.5\,\mathrm{s}$, $F = L/2$, heartbeat interval 1 s and silence timeout 2.5 s. The task becomes claimable at r when its copy of the last renewal expires (Proposition 4). The failure vote "v" and any certificate are not on this path; a certificate may never form.

F. Receiver-local task selection

Each idle UAV ranks the objectives it can claim, $A_i(t)$ (not complete at i, with every known incompatible claim expired or released), by an additive cost evaluated only from its own state and delivered evidence:

$$\mathrm{cost}_i(j) = w_{\mathrm{d}}\|p_i - p_j\| + w_{\mathrm{r}}\, r_i(j) + w_{\mathrm{c}}\, \kappa_i(j) + w_{\mathrm{o}}\, \varrho_i(j), \tag{4}$$

with ties broken by task identifier. Here r_i and ϱ_i are resource and role terms, and $\kappa_i(j) = (N-1) - \hat{d}_i(p_j)$ is a communication term: $\hat{d}_i$ counts peers whose *last delivered, still HEARD* position lies within range of p_j. The term never reads $G(t)$. By default only distance is weighted. The original design note asked for a richer utility (information gain minus travel, energy, communication and operational risk), on the observation that "the closest UAV is not always the best UAV"; Eq. (4) implements the travel and communication parts and keeps resource and role as validated extension points. The live mission expresses one role preference, holding physical vehicles in reserve, as a receiver-local delay rule (Section VII-D).

Algorithm 1 is the task-control round run at every event boundary. Choices are batched from pre-mutation views, so agent traversal order cannot silently allocate a task. A claim exists before an intent is bound, and a single predicate, `task_is_executable`, gates motion, renewal and completion. It requires that the UAV is responsive, that its cached intent names the task, and that its own ledger reports OWNED_BY_SELF.

G. Composed EFSM kernel and the mission contract

Each UAV composes small machines instead of one product automaton (Table II). Every transition is a pure function

$$\delta\big((q, x), e, \pi\big) \mapsto (q', x', \mathcal{E}, g, \mathit{reason}),$$

Algorithm 1 Receiver-local task-control round at time t

for each participating UAV i **do**
  advance the lease clock of ledger i to t
  stand down unless ledger i still owns intent j_i
  if $j_i \neq \perp$ **and** $a_i(c_i, t) \geq F$ **then**
    renew: create $(j_i, i, e+1)$
  end if
end for
repeat
  $P \leftarrow \{(i, \arg\min_{j \in A_i(t)} \mathrm{cost}_i(j)) : i \text{ idle}\}$
  create a claim $(j, i, e_{ij}+1)$ for every $(i,j) \in P$
  flood new immutable evidence over $G(t)$
  each i reconciles contests; losers release
  each idle i owning its claim binds intent $j_i \leftarrow j$
until no claim, stand-down or activation occurred

TABLE II
COMPOSED RECEIVER-LOCAL EFSMS (PER UAV)

Machine	Count	States	Driven by
Contact	per peer	UNKNOWN, ACTIVE, DEGRADED, LOST, RECOVERING	direct-hop receipts; local time
Peer avail.	per peer	SILENT, HEARD, DECLARED_FAILED	first-hand heartbeat; silence; valid certificate
Task owner.	per task	six states of Fig. 3	local ledger view (conformance)
Coordin. mode	one	COOPERATIVE, DEGRADED, LOCAL_AUTONOMY, RECONCILING	composed contact states; visible contests; new remote evidence
Mission phase	one	BRIEFED, EXECUTING, HOLDING, ABORTED, COMPLETE	held contract version; coordination mode; open objectives

of finite state q, typed extended variables x, a typed local event e and a policy π. It returns the next configuration, a list of requested effects $\mathcal{E}$ and the guard g and reason that justified it. Reducers never move a vehicle, send a packet or read observer state. Effects are data, and in the current prototype the coordination mode is advisory: execution authority remains the claim ledger. Contact recovery requires receipts at *distinct* local times, so a same-timestamp burst cannot manufacture stability. Contact evidence is attributed to the immediate forwarder, not to a message's logical origin.

The operator's intent is a versioned `MissionContract`: objective priorities, a minimum-participants floor and a degradation policy $\in \{\mathsf{CONTINUE}, \mathsf{HOLD}, \mathsf{ABORT}\}$. Under HOLD, a UAV whose coordination has stayed degraded for the contracted grace period (default 5 s) holds; it resumes after coordination has been stable for the resume window (default 2 s). It is deliberately not a plan: nothing in it says who does what, because the operator is not on the network when the partition happens. Amendments flood as ordinary evidence and supersede strictly by version, so a partition splits intent exactly as it splits everything else. A UAV holding no contract is unconstrained rather than forbidden: *a contract can only add restraint*.

V. IMPLEMENTATION

The prototype, `eudis-swarm`, is a Python 3.11+ package of about 20k lines of source and 13k lines of tests (531 test functions), with *no* runtime dependencies in its core. MAVLink bindings, NumPy (for the quantum planner) and the Streamlit dashboard are optional extras. The simulator has no wall-clock time, no unseeded randomness and no order-dependent iteration. Every event boundary executes a fixed sequence: claim-authorised motion, fault injection, graph recomputation, peer-freshness ageing, heartbeat delivery, failure votes and declarations, observer-side recovery effects, the task-control fixed point of Algorithm 1, completion, and trace capture. The same seeds therefore produce byte-identical outputs, which a test asserts. Continuous integration runs lint, strict type checking, tests with an 85% branch-coverage floor and a package build on Python 3.11 and 3.13. Trace schema 3 records, for every UAV, its peer evidence and age, its ownership view per task, its contract posture and every EFSM state change with guard and reason. A playback dashboard reconstructs belief panels only from those per-agent records, and an architecture test fails the build if it imports a module that carries authoritative world state.

Centralized baseline. For paired comparison, a selectable mode replaces the claim round with one coordinator that has a consistent global view. Each round it assigns globally nearest idle-UAV/task pairs greedily, refuses a task whose lease is still valid at the target UAV, and releases a failed UAV's task only when a quorum declaration is established. This is a deliberately strong baseline in what it *knows*; it differs from the distributed mode only in that reassignment waits for agreement.

VI. EVALUATION IN DETERMINISTIC SIMULATION

A. Setup and metrics

Table III lists the defaults. Every comparison is *paired*: the distributed and centralized runs share seed, task placement, range, jammer and fault schedule, and differ in one setting. Seeds vary task positions. Durations are *logical* seconds under a point-mass motion model, not flight times (Section VII-G). Coordination metrics are evaluator-computed ground truth, obtained by reading every replica's private store at one instant, and are integrated over logical time in task-seconds. *Stranded* (dropped-ownership) time counts a previously owned, incomplete task with no responsive self-claimant. *Double-booked* (concurrent-ownership) time counts a task with more than one local self-claimant. The latter can include a stopped aircraft that still holds a claim, so it is an upper bound on, not a count of, duplicated flight. *Undetected* losses are aircraft that physically stopped and were never declared.

B. A split brain, and its resolution

Fig. 5 traces the deterministic 2+2 demonstration ($F = 1.0\,\mathrm{s}$, $L = 3.0\,\mathrm{s}$). UAV 1 claims Task 19 at $t = 0$; the cross

TABLE III
DEFAULT SIMULATION PARAMETERS

Parameter	Value
UAVs N / tasks $\lvert\mathcal{J}\rvert$	4 / 20
Area; speed	100×100 units; 10 units/s
Time step; arrival tolerance	0.25 s; 0.75 units
Heartbeat interval; silence timeout	1.0 s; 2.5 s
Lease L; freshness F	2.5 s; $L/2$
Communication range R	130 (swept 30–130)
Physical failure	UAV 2 at $t = 4.0$ s
Region jammer (when used)	centre $(40, 50)$, radius 15, $t \in [3, 8)$ s
Mission budget	300 s
Seeds per condition	20 consecutive

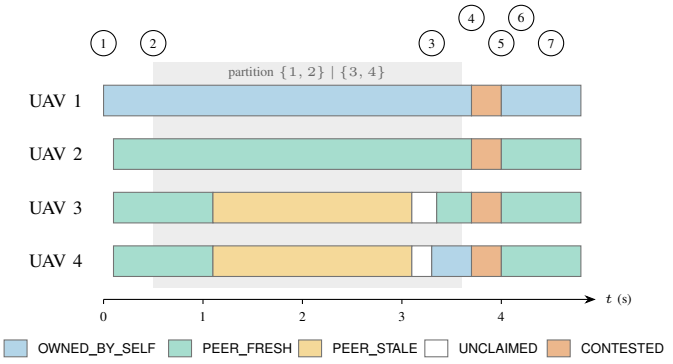


Fig. 5. Each UAV's own view of Task 19 in the 2+2 split-brain demonstration. Events: (1) UAV 1 claims; (2) partition; (3) UAV 4 claims after its copy expired (split brain); (4) reconnection makes both claims visible; (5) independent reconciliation selects UAV 1; (6) UAV 4's release delivered; (7) UAV 4 claims Task 29.

links $(1,3),(1,4),(2,3),(2,4)$ are then cut. UAV 1 keeps renewing inside $\{1,2\}$, but its renewals cannot reach $\{3,4\}$, whose copies go stale at $t \approx 1.1$ and expire at $t \approx 3.1$. UAV 4 then legitimately claims Task 19 itself: both UAV 1 and UAV 4 report OWNED_BY_SELF. On reconnection, all four replicas see both claims and report CONTESTED; at $t = 4.0$ each selects UAV 1 independently, UAV 4 releases, all replicas converge on one owner, and UAV 4 claims Task 29 at $t = 4.5$ without any restart. In the full default mission (20 tasks, UAV 2 fails at $t = 4$ while holding Task 19), all 20 tasks complete in 17.25 s. A survivor activates Task 19 at $t = 10.75$ s, after its own copy of the lease has expired.

C. Identical awareness, opposite outcomes

We swept the communication range from 30 to 130 (11 values, 20 seeds, both modes; 440 runs) with the region jammer active and UAV 2 failing at $t = 4$ s (Fig. 6). **The centralized coordinator failed 68 of 220 missions; the distributed protocol failed none.** A failed central mission is not a partial one: it completes 19 of 20 tasks and then spends the rest of its 300 s budget holding the last task assigned to an aircraft that has stopped (297.4 task-seconds stranded at range 30, against 1.8 for the distributed mode). Detection, however, is *identical* in the two modes: in all 220 seed pairs the two modes agree on whether the loss was ever declared (undetected in 20/20 seeds at ranges 30–40, 18 at 50, 8 at 60, 2 at 70, none from 80). The 68 central failures are exactly the 68 runs in which the loss went undetected. What separates the modes is not what they know but whether acting on it required agreement. Above range 60 the distributed mode is flat (12.43 s, 1.7 task-seconds stranded, 0.04 double-booked), so its behaviour stops depending on connectivity well before central control stops failing.

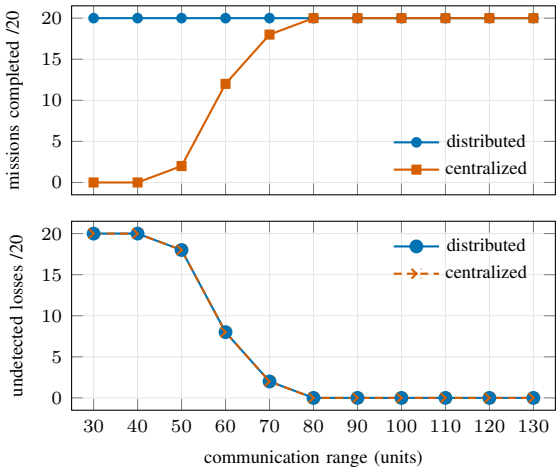


Fig. 6. Range sweep with a region jammer and a fail-stop (440 paired runs). Top: mission completion. Bottom: aircraft that stopped and were never declared. The curves coincide: identical awareness, opposite outcomes.

TABLE IV
QUORUM FLOOR: ONE AIRCRAFT STOPS

UAVs	Mode	Declared	Undetected	Tasks	Mission
2	centralized	0	1	2/3	never finishes
2	distributed	0	1	3/3	10.75 s
3	centralized	1	0	3/3	completed
3	distributed	1	0	3/3	completed

D. The quorum floor: a loss that cannot be declared

A robustness sweep over 432 configurations produced no crashes but 108 incomplete missions, *all* centralized. The minimal case is two aircraft (Table IV; presets `demo-quorum-floor` and `baseline-quorum-floor`). By Proposition 1, the survivor is the only witness and can never declare its peer. Both modes are equally blind. Central reassignment waits for a declaration that cannot arrive and never finishes, while the distributed survivor re-claims the work when its lease copy expires. A third aircraft restores the quorum and both modes recover, which confirms that the floor is about witness count, not the control model. In short, *consensus-free recovery keeps working in exactly the conditions that make consensus impossible.*

E. The price of availability, and the operator's dial

Availability is not free. In a 480-run lease sweep ($L \in [0.5, 6]$ s, range 50, jammer, no fail-stop; Fig. 7a) all runs completed, but *the two modes fail in opposite directions.*

Central control never double-booked in this sweep, but it handed work to aircraft it could not reach: 4.79 task-seconds stranded on average, falling by a factor of 864 as the lease grows. Replicated ownership stranded nothing and paid 0.29 task-seconds of double booking during the partition. That cost falls from 0.51 to 0.25 and then plateaus, because a replica that cannot hear the owner re-claims once its own copy expires, however long the lease. Zero central double booking is an observation for this configuration, not a structural guarantee: the fleet study below found brief overlap involving a stopped UAV.

The operator's degradation policy moves the mission along this trade-off (420 runs; Fig. 7b,c). HOLD removes 64% of double booking (0.323→0.116 task-seconds) and eliminates it from range 70 upwards, at the cost of 59% longer missions (10.71→17.06 s). An uncontracted swarm and CONTINUE agree to three decimal places on every metric across 280 runs, which confirms by measurement the invariant that a contract only adds restraint. Which policy is right depends on the mission, which is why the choice belongs to the operator rather than to the code.

F. The detection floor: a cliff for one layout, a gradient for many

Failure detection needs a quorum of peers that can still hear one another, and nothing guarantees that one exists. For the default task layout (seed 2026, no jammer), detection fails as a cliff: the loss is declared at every tested range from 76 to 130 and at none from 75 down, yet all 20 tasks still complete because leases expire on their own. Across 20 task-placement seeds the same detector degrades gradually instead (18/20 losses undetected at range 50, 8 at 60, 2 at 70, none from 80), with or without the jammer. Both shapes are real; the cliff belongs to one geometry. Either way, *no mission-level number reveals that an aircraft is gone*, which is precisely what makes undetected loss dangerous and why we report it as its own metric.

G. Fleet size, network regime, layout and fault timing

A paired fleet/network study (240 runs; Table V) scales the area side as $100\sqrt{N/4}$ with two tasks per UAV, fails UAV 2 at $t = 4$ s and caps missions at 30 s. In the sparse regimes the distributed mode finished when its paired central run timed out in 18/20 (four UAVs) and 19/20 (six UAVs) seeds, never the reverse. Central control showed small but non-zero overlap in four high-range runs. A trace shows Task 5 claimed both by a responsive UAV and by the stopped UAV 2: claim overlap after reassignment, not two aircraft flying.

A second study (960 runs; Table VI) adds a centred-ring starting layout and moves the fault to $t = 2$ or $t = 6$ s. The distributed mode completed all 480 of its runs and the centralized mode 337. The gap depends strongly on geometry and fault time: an early fault in the default layout gave 0/20 central completions in every sparse condition, while a late fault on the ring often gave 20/20. The ring also produces fewer components in sparse runs (mean maximum 1.43 vs. 3.21). Late-fault ring runs need a caveat: in 81 of 960 rows the mission finished *before* the scheduled fault. Restricted to the 75 pairs in which both modes actually experienced the fault, both modes completed every run except under six-UAV sparse plus jammer, where the distributed mode completed 14/14 against 12/14. The jammer raised mean distributed overlap from 0.33 to 0.94 task-seconds (central: 0.03→0.02).

TABLE V
PAIRED FLEET AND NETWORK COMPARISON (20 SEEDS PER CELL; MEANS IN TASK-SECONDS; D = DISTRIBUTED, C = CENTRALIZED)

		Complete		Stranded		Overlap	
N	Network	D	C	D	C	D	C
4	high range	20	20	0.03	6.65	0.00	0.07
4	sparse	20	2	0.28	27.38	0.72	0.00
4	sparse+jam	20	2	0.60	27.40	1.73	0.00
6	high range	20	20	0.03	11.25	0.00	0.03
6	sparse	20	1	0.07	32.73	0.12	0.00
6	sparse+jam	20	1	0.17	32.73	0.78	0.00

TABLE VI
LAYOUT AND FAULT-TIME SENSITIVITY: MISSIONS COMPLETED OUT OF 20, D / C (960 PAIRED RUNS)

N	Layout	Fault	High range	Sparse	Sparse+jam
4	default	2 s	20 / 20	20 / 0	20 / 0
4	default	6 s	20 / 20	20 / 19	20 / 16
4	ring	2 s	20 / 20	20 / 10	20 / 10
4	ring	6 s†	20 / 20	20 / 20	20 / 20
6	default	2 s	20 / 20	20 / 0	20 / 0
6	default	6 s	20 / 20	20 / 18	20 / 18
6	ring	2 s	20 / 20	20 / 4	20 / 4
6	ring	6 s†	20 / 20	20 / 20	20 / 18

† Some runs finished before the scheduled fault; see text for the fault-exposed subset.

VII. LIVE MISSION ON ARDUPILOT SITL

For the EUDIS Defence Hackathon (Challenge 2, *Swarm Coordination and Autonomous Operations*), the claim protocol was taken out of the simulator and used to fly a mission on ArduPilot [39]. One ground-node laptop runs one receiver-local agent per aircraft and commands each autopilot in GUIDED mode over MAVLink [40]. Vehicles may be SITL, physical, or both, in one local frame georeferenced at the USRL Orounda airfield. Inter-UAV links are emulated with the same `CommunicationGraph`, fed by real telemetry positions; claim gossip, leases and lowest-identifier reconciliation are the simulator's own code. This harness runs on wall-clock time and is *not* deterministic, so a live run is reported as an observation, never as a measured result.

A. Ground node and safety envelope

Aircraft have one of two roles: ISR (search and detection) or LM (loitering-munition role for engagement; here an ordinary multirotor whose release is simulated). Each role has an altitude band: ISR 5–25 m and LM 30–60 m, under a 60 m ceiling and a 400 m operating radius. The two roles are therefore vertically separated by construction. Every setpoint is checked against the vehicle's band and radius before it is

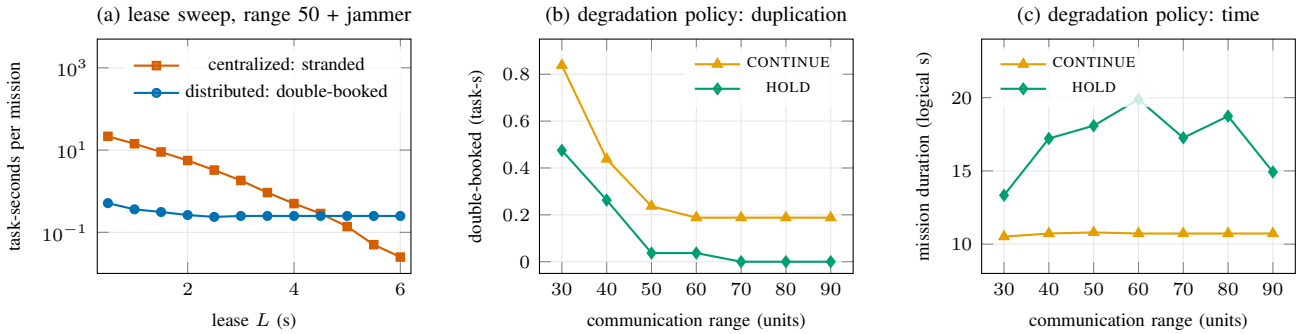


Fig. 7. The price of availability. (a) Central control strands work it cannot reach; replicated ownership instead double-books briefly during a partition (480 runs, all completed). (b,c) The operator's HOLD policy trades duplication for time (420 runs, 20 seeds per point; uncontracted runs coincide with CONTINUE and are not drawn). Lines between points are drawn for readability only.

stored. The ground node never arms a physical aircraft: a safety pilot takes off and hands over by selecting GUIDED. Any mode change the ground node did not command is treated as a final takeover, after which the vehicle is never commanded again in that run. Commands are re-sent until they take effect, heartbeats go out at 1 Hz so that autopilot failsafes fire only if the ground node is really gone, telemetry older than 2 s is never treated as current, and an interrupt returns every vehicle to launch. The autopilot's own geofence and failsafes remain the safety net.

B. Search as claimable sectors

The operator's convex search area (default 240×160 m) is cut into strips. Each strip is a task in the claim protocol: ISR aircraft claim sectors and fly boustrophedon lanes [27] at their own altitude, spaced so that consecutive camera footprints overlap by 20% (a Pi Camera v2 sees about 24 m across at 20 m, roughly 3.8 cm per pixel). An ISR publishes completion at the last waypoint. Every participating aircraft relays evidence.

C. Perception, geolocation and target discovery

A detection is a pixel (u, v) in a $W \times H$ frame from a nadir camera with fields of view θ_h, θ_v. At height h it maps to the ground offset

$$\Delta_{\text{right}} = \tfrac{2u-W}{W}\, h \tan\tfrac{\theta_h}{2}, \quad \Delta_{\text{fwd}} = \tfrac{H-2v}{H}\, h \tan\tfrac{\theta_v}{2}, \qquad (5)$$

rotated by the aircraft heading. Attitude is assumed level; at search speed a few degrees of pitch move the estimate by $h\tan(\text{pitch})$, a metre or two at 20 m. Each ISR fuses its sightings into local tracks and publishes an immutable *target report*, which floods like any other evidence and becomes a new engagement task. Two ISRs may report one object before hearing each other. Among the reports a UAV knows, one lying within 10 m of a report with a lower task identifier is treated as its duplicate, so every receiver with the same evidence engages the same canonical report. SITL ISRs use a synthetic camera with misses and pixel noise that never reveals target identity. Physical ISRs stream version-1 JSON-Lines observations over UDP from an on-board detector, geolocated with the aircraft's telemetry *at the frame's timestamp*, which requires the on-board computer's clock to be synchronised with the ground node. Camera health is evidence too: a camera that reports *stalled*, or sends nothing for 3 s, is shown as silent, and silence never counts as an empty scene.

D. Human-confirmed engagement

Each target report is an engagement task claimed by LMs through the same protocol. The owner transits at 40 m and *holds 30 m short until the operator confirms* on the map; *Abort* withdraws the target for the rest of the run. After confirmation the LM descends to 30 m over the aim point, dwells 3 s and performs a *simulated* release, but only if it still owns the claim at that instant. The release publishes completion, and the LM returns to launch, spent. Nothing leaves the aircraft; on a physical LM the release pulses an LED or buzzer on an autopilot relay. Both operator decisions attach to the *object*, so a duplicate report inherits them. If the LM owning a confirmed target fails, its claim expires after the lease and the next free LM takes over under the same confirmation. Physical LMs act as a reserve: a reserve claims a target only after it has stayed unclaimed for 8 s in its own view, a receiver-local role preference that keeps the expensive asset back while expendable ones are free.

E. Advisory QAOA assignment planner

The challenge asks for optimisation using quantum simulation. When the set of idle LMs $\mathcal{L}$ or open targets $\mathcal{T}$ changes, the ground node solves the assignment as a QUBO [35] over $x_{\ell t} \in \{0, 1\}$. For $|\mathcal{L}| \geq |\mathcal{T}|$,

$$E(x) = \sum_{\ell,t} d_{\ell t} x_{\ell t} + P \sum_{t} \Big(1 - \sum_{\ell} x_{\ell t}\Big)^2 + P \sum_{\ell} \sum_{t<t'} x_{\ell t} x_{\ell t'}, \qquad (6)$$

with straight-line distances $d_{\ell t}$ and $P = 2 \max d + 1$; the roles of the two constraints swap when targets outnumber LMs. We simulate the quantum alternating operator ansatz [33] on a NumPy statevector. The initial state is a product of W-states, one per target (an equal superposition of "exactly one LM").

TABLE VII
QAOA PLANNER VERSUS EXACT OPTIMUM (20 SEEDED INSTANCES PER SIZE, $p = 3$). SIZE: LMS × TARGETS. OPT.: READOUT EQUALS THE EXACT OPTIMUM. P_{opt}: PROBABILITY QAOA PLACES ON THE OPTIMUM. BASELINES: UNIFORM OVER THE REACHABLE ONE-HOT SUBSPACE (UNIF.) AND OVER VALID ASSIGNMENTS (VALID). TIMES ON A LAPTOP CPU.

Size	Qubits	Opt.	P_{opt}	Unif.	Valid	Time (s)
2×2	4	20/20	0.514	0.250	0.500	0.01
3×2	6	20/20	0.208	0.111	0.167	0.04
4×2	8	20/20	0.107	0.063	0.083	0.06
3×3	9	20/20	0.085	0.037	0.167	0.08
5×2	10	19/20	0.056	0.040	0.050	0.11
4×3	12	19/20	0.034	0.016	0.042	0.28

Each of $p = 3$ layers applies the cost phase $e^{-i\gamma_k \tilde{E}}$ followed by an XY ring mixer [34] inside each target's group, which never leaves the one-hot subspace. $\tilde{E}$ is E normalised over that subspace. Angles follow a linear ramp chosen by grid search and are then refined coordinate by coordinate. The answer is the best feasible bitstring among the 16 most probable, as repeated measurement would give. A transverse-field mixer was tried first and abandoned: on the 4×3 instance only 24 of 4,096 bitstrings are valid, and it placed about 0.1% probability on the optimum.

The plan is *advice*. Each LM adds 50 m to every target except its recommended one, and the claim protocol still decides who flies. The exact optimum is computed by enumeration alongside every solve; classically, this assignment problem is solvable in polynomial time by the Hungarian method [41]. Table VII benchmarks 120 seeded random instances (LMs on the launch row, targets in the search area). QAOA's readout matched the exact optimum in 118; the two misses cost 1.1% and 2.4% more than the optimum. The probability it places on the optimum, however, is only 1.4–2.3 times that of the uniform initial state, and it is *comparable to sampling a valid assignment uniformly at random* (lower at 3×3 and 4×3). At these sizes the 16-candidate readout covers much of the reachable subspace (16 of 64 states at 12 qubits), so the match rate reflects classical post-selection as much as the ansatz. There is no quantum advantage here, and we claim none. The value is the formulation and an honest integration that keeps the quantum step off the critical path.

F. Observed runs

All runs were on one development laptop with eight ArduCopter SITL instances, and each was flown once. *Ring flight:* every vehicle reached its ring point within 3 m at its layer altitude and landed at its launch point. *Search with a fail-stop:* with `isr_04` killed at 60 s while owning sector S3, two ISRs claimed S6 at once and the lower identifier kept it; the first ISR free after S3's lease had expired took it over at 89.9 s; all six sectors were searched exactly once by 163 s. *Full demonstration* (Table VIII, Fig. 8): an ISR inside an emulated jam zone detected the target while isolated, and its report flooded out when its links returned. The first LM to engage was fail-stopped by design; after its lease expired a second LM took the target, held short until the operator confirmed, and released 0.3 m from its aim point (the geolocated estimate) and 0.1 m from the object's true position. The reserve LM engaged last, once the others were spent. All six sectors were searched, all three targets were eliminated, and all eight aircraft landed.

TABLE VIII
EVENT LOG EXCERPT, FULL SITL DEMONSTRATION (ONE RUN, OBSERVED)

t (s)	Vehicle	Event
76.0	isr_02	enters active jam zone
81.2	isr_02	detects T20 while jammed
81.2	planner	QAOA advice lm_02→T20 (= exact optimum)
81.6	isr_02	links restored; report floods out
81.6	lm_02	engages T20; fail-stop injected
93.6	lm_03	engages T20 after lm_02's lease expired
103.5	lm_03	holds 30 m short of T20
130.3	operator	confirms T20 on the map
140.8	lm_03	simulated release, 0.3 m from aim point
197.8	lm_01	reserve engages T11 (others spent)
257.3	lm_01	simulated release, 0.3 m from aim point

TABLE IX
POINT-MASS PREDICTION VS. ARDUCOPTER SITL, 60 M LEG (6 LEGS, ONE VEHICLE; SITL IS NOT REPRODUCIBLE, SO RANGES ARE SHOWN)

	Min	Max	Mean
Duration / model prediction	1.53	1.76	1.71
Worst position error during leg (m)	25.25	37.26	34.43
Arrival error (m)	1.40	1.99	1.67

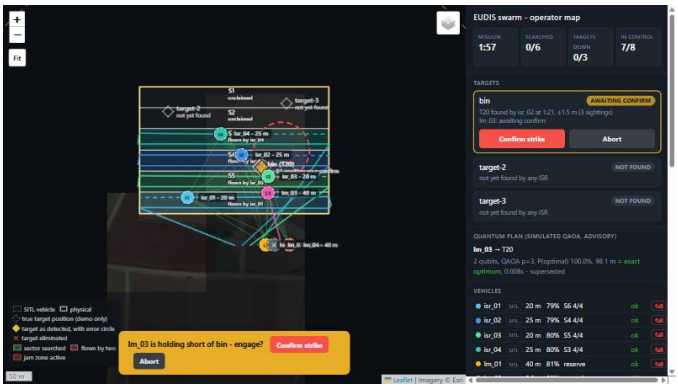
G. How wrong is the point-mass model?

The simulator predicts motion with a point mass: constant speed, instant acceleration, exact arrival. We flew the same 60 m leg six times on one ArduCopter SITL vehicle, giving the model the autopilot's own 10 m/s so that it had its best case (Table IX). Real legs took 1.71 times longer than predicted. Peak divergence, 34 m on average, occurred at 5.8 s, exactly when the model says the aircraft has arrived and stopped, while the real one was still a third of the way back. Arrival error was small (1.7 m). This is why all simulated durations are reported as logical seconds. Its effect on coordination decisions has not yet been measured.

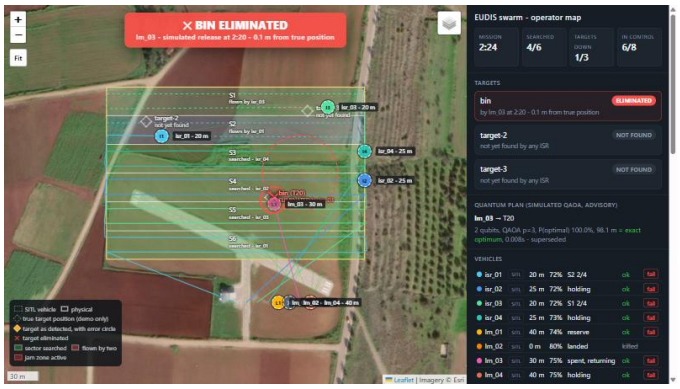
VIII. DISCUSSION AND LIMITATIONS

What the evidence supports. Within a deterministic, two-dimensional, point-mass simulation with an abstract range/jammer link model, receiver-local leases recover work in conditions where declaration-gated reassignment cannot. The difference is not explained by awareness, because detection is identical. Replicated ownership pays with transient double booking, and the operator can trade it against time.

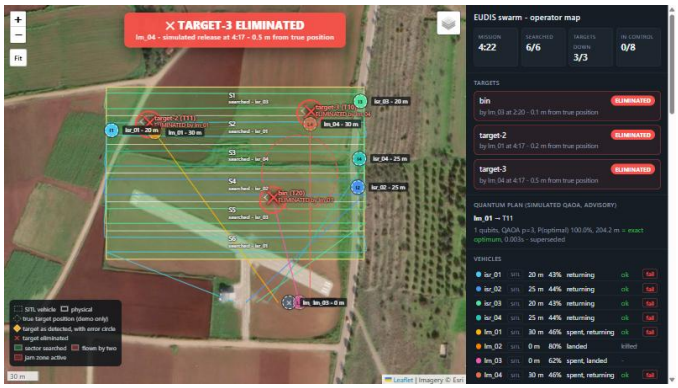
What it does not support. (i) *RF:* the link model is analytical and uncalibrated, and the threat sources concern GNSS, not inter-vehicle links. (ii) *Absolute time:* durations are logical seconds under a motion model whose error we measured at 1.7×. (iii) *Generality:* the results cover a few scenario



(a) Holding short: the LM awaits *Confirm strike*.



(b) First target eliminated (release 0.1 m from true position).



(c) Mission complete: 6/6 sectors searched, 3/3 targets eliminated, all aircraft returned.

Fig. 8. The operator map during the eight-vehicle SITL mission at USRL Orounda (imagery © Esri, map library Leaflet [42]). Sectors are tinted by the UAV flying or having searched them, red if two fly one at once. Dashed diamonds mark true target positions for demonstration only; no UAV sees them. Solid diamonds with error circles are geolocated estimates. The side panel shows the latest advisory QAOA plan. *Confirm* and *Abort* are the only controls by which the page can affect the mission.

families (4–6 UAVs, two layouts, one jammer geometry). The completion gap is highly sensitive to geometry and fault timing (Table VI). (iv) *Baseline:* our centralized coordinator is one specific design, declaration-gated and greedy; a coordinator that reassigned on silence alone would recover faster but would also reassign work from healthy, merely partitioned aircraft, the failure mode this work is built to avoid. (v) *Faults:* only fail-stop is modelled; Byzantine behaviour, identity forgery, asymmetric links, latency, bandwidth and simultaneous failures are out of scope. Flooding retains seen-state and can grow as $O(mN^2)$ for m messages. (vi) *EFSMs:* requested effects have no dispatcher yet, and the coordination mode is advisory. (vii) *Live mission:* the ground node hosts all agents in one process with emulated inter-UAV links. Live runs are single observations. Fleet telemetry does not yet drive the deterministic simulator through its vehicle interface, and no physical aircraft or live perception stream has yet flown the mission end to end. (viii) *QAOA:* no quantum advantage at this size; its readout is substantially classical post-selection (Section VII-E).

Responsible use. The engagement task in the live mission is a demonstration of human authority in a coordination protocol, not a weapon. The release is simulated and nothing leaves the aircraft. Every engagement requires explicit operator confirmation per object, *Abort* is final, a release requires continuing ownership at that instant, and any pilot mode change permanently removes the swarm's authority over that vehicle. The software is a research prototype; it is not flight-certified and must not be used to command real vehicles outside an authorised, supervised test range.

IX. Conclusion

Silence is ambiguous, and in a contested RF environment it is the normal case. We built a swarm-coordination protocol around that fact: each UAV acts only on evidence it has received, ownership is a lease rather than a verdict, conflicts created by partition are allowed and reconciled deterministically, and failure declarations inform rather than gate recovery. In paired simulations the distributed and centralized modes see exactly the same losses, yet only the consensus-free mode keeps completing missions once the network is sparse, jammed or too small for a quorum. The cost is transient double booking, which the operator can buy down with a HOLD policy. The same protocol coordinates search and human-confirmed engagement on eight autopilot SITL vehicles. Next steps are to drive the simulator from fleet telemetry, to fly the

mixed physical/SITL mission at the airfield, to run agents in separate processes over real radios, to dispatch EFSM effects into role and relay behaviour, and to replace the point-mass model with a calibrated airframe model.

Acknowledgment

The author thanks the organisers of the EUDIS Defence Hackathon 2026 (Cyprus) and the other members of Team Teleforce; the on-board perception pipeline for the physical ISR aircraft, based on VERA-MOT, is the work of a fellow team member. The views expressed are the author's own and are not endorsed by the hackathon organisers or partners.

References

[1] M. Brambilla, E. Ferrante, M. Birattari, and M. Dorigo, "Swarm robotics: A review from the swarm engineering perspective," *Swarm Intelligence*, vol. 7, no. 1, pp. 1–41, 2013.

[2] S.-J. Chung, A. A. Paranjape, P. Dames, S. Shen, and V. Kumar, "A survey on aerial swarm robotics," *IEEE Transactions on Robotics*, vol. 34, no. 4, pp. 837–855, 2018.

[3] European Union Aviation Safety Agency, "GNSS outages and alterations," https://www.easa.europa.eu/en/domains/air-operations/global-navigation-satellite-system-outages-and-alterations, 2026, affected-FIR tables, consulted 6 September 2026.

[4] ——, "Safety information bulletin SIB 2022-02R4: Global navigation satellite system outage and alterations leading to navigation/surveillance degradation," https://www.easa.europa.eu/en/newsroom-and-events/news/easa-updates-safety-information-bulletin-gnss-interference, 2026, issued 3 July 2026.

[5] O. Osechas, F. Fohlmeister, T. Dautermann, and M. Felux, "Impact of GNSS-band radio interference on operational avionics," *NAVIGATION: Journal of the Institute of Navigation*, vol. 69, no. 2, 2022.

[6] M. J. Fischer, N. A. Lynch, and M. S. Paterson, "Impossibility of distributed consensus with one faulty process," *Journal of the ACM*, vol. 32, no. 2, pp. 374–382, 1985.

[7] S. Gilbert and N. Lynch, "Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services," *ACM SIGACT News*, vol. 33, no. 2, pp. 51–59, 2002.

[8] T. D. Chandra and S. Toueg, "Unreliable failure detectors for reliable distributed systems," *Journal of the ACM*, vol. 43, no. 2, pp. 225–267, 1996.

[9] B. P. Gerkey and M. J. Matarić, "A formal analysis and taxonomy of task allocation in multi-robot systems," *The International Journal of Robotics Research*, vol. 23, no. 9, pp. 939–954, 2004.

[10] H.-L. Choi, L. Brunet, and J. P. How, "Consensus-based decentralized auctions for robust task allocation," *IEEE Transactions on Robotics*, vol. 25, no. 4, pp. 912–926, 2009.

[11] L. Lu *et al.*, "Communication-aware distributed task allocation for synchronous multi-UAV collaboration," *Aerospace Science and Technology*, p. 113589, 2026.

[12] İ. Bekmezci, O. K. Sahingoz, and Ş. Temel, "Flying ad-hoc networks (FANETs): A survey," *Ad Hoc Networks*, vol. 11, no. 3, pp. 1254–1270, 2013.

[13] S. Hayat, E. Yanmaz, and R. Muzaffar, "Survey on unmanned aerial vehicle networks for civil applications: A communications viewpoint," *IEEE Communications Surveys & Tutorials*, vol. 18, no. 4, pp. 2624–2661, 2016.

[14] G. DeCandia, D. Hastorun, M. Jampani, G. Kakulapati, A. Lakshman, A. Pilchin, S. Sivasubramanian, P. Vosshall, and W. Vogels, "Dynamo: Amazon's highly available key-value store," in *Proceedings of the 21st ACM Symposium on Operating Systems Principles (SOSP)*, 2007, pp. 205–220.

[15] W. Vogels, "Eventually consistent," *Communications of the ACM*, vol. 52, no. 1, pp. 40–44, 2009.

[16] M. Shapiro, N. Preguiça, C. Baquero, and M. Zawirski, "Conflict-free replicated data types," in *Stabilization, Safety, and Security of Distributed Systems (SSS)*, ser. Lecture Notes in Computer Science, vol. 6976. Springer, 2011, pp. 386–400.

[17] A. Demers, D. Greene, C. Hauser, W. Irish, J. Larson, S. Shenker, H. Sturgis, D. Swinehart, and D. Terry, "Epidemic algorithms for replicated database maintenance," in *Proceedings of the 6th ACM Symposium on Principles of Distributed Computing (PODC)*, 1987, pp. 1–12.

[18] C. G. Gray and D. R. Cheriton, "Leases: An efficient fault-tolerant mechanism for distributed file cache consistency," in *Proceedings of the 12th ACM Symposium on Operating Systems Principles (SOSP)*, 1989, pp. 202–210.

[19] J. A. Tran *et al.*, "SwarmDAG: A partition tolerant distributed ledger protocol for swarm robotics," *Ledger*, vol. 4, 2019.

[20] Algolysis, CMMI, and CYENS, "CHARISMA: Heterogeneous unmanned vehicles for maritime surveillance," https://projects.algolysis.com/charisma/, 2026, project page, consulted 3 September 2026.

[21] L. Lamport, "The part-time parliament," *ACM Transactions on Computer Systems*, vol. 16, no. 2, pp. 133–169, 1998.

[22] D. Ongaro and J. Ousterhout, "In search of an understandable consensus algorithm," in *Proceedings of the USENIX Annual Technical Conference*, 2014, pp. 305–319.

[23] Y. K. Lopes, S. M. Trenkwalder, A. B. Leal, T. J. Dodd, and R. Groß, "Supervisory control theory applied to swarm robotics," *Swarm Intelligence*, vol. 10, pp. 65–97, 2016.

[24] E. Santamaría Barnadas, "Formal mission specification and execution mechanisms for unmanned aircraft systems," Ph.D. dissertation, Universitat Politècnica de Catalunya, 2010.

[25] A. A. Tziola and S. G. Loizou, "A formal framework for multi-agent task planning," *IEEE Transactions on Automatic Control*, vol. 71, no. 1, pp. 630–637, 2026.

[26] Anduril Industries, "Lattice developers: Tasks overview," https://developer.anduril.com/guides/tasks/overview/, 2026, consulted 3 September 2026.

[27] H. Choset, "Coverage of known spaces: The boustrophedon cellular decomposition," *Autonomous Robots*, vol. 9, no. 3, pp. 247–253, 2000.

[28] E. Galceran and M. Carreras, "A survey on coverage path planning for robotics," *Robotics and Autonomous Systems*, vol. 61, no. 12, pp. 1258–1276, 2013.

[29] S. G. Loizou and C. C. Constantinou, "Multi-robot coverage on dendritic topologies under communication constraints," in *Proceedings of the 55th IEEE Conference on Decision and Control (CDC)*, 2016, pp. 43–48.

[30] S. Papaioannou *et al.*, "Distributed search planning in 3-D environments with a dynamically varying number of agents," *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, 2023.

[31] M. Cao *et al.*, "Cooperative aerial robot inspection challenge: A benchmark for heterogeneous multi-UAV planning and lessons learned," arXiv:2501.06566, 2025.

[32] E. Farhi, J. Goldstone, and S. Gutmann, "A quantum approximate optimization algorithm," arXiv:1411.4028, 2014.

[33] S. Hadfield, Z. Wang, B. O'Gorman, E. G. Rieffel, D. Venturelli, and R. Biswas, "From the quantum approximate optimization algorithm to a quantum alternating operator ansatz," *Algorithms*, vol. 12, no. 2, p. 34, 2019.

[34] Z. Wang, N. C. Rubin, J. M. Dominy, and E. G. Rieffel, "XY mixers: Analytical and numerical results for the quantum alternating operator ansatz," *Physical Review A*, vol. 101, no. 1, p. 012320, 2020.

[35] A. Lucas, "Ising formulations of many NP problems," *Frontiers in Physics*, vol. 2, p. 5, 2014.

[36] K. Blekos *et al.*, "A review on quantum approximate optimization algorithm and its variants," *Physics Reports*, vol. 1068, pp. 1–66, 2024.

[37] K.-C. Chen *et al.*, "Resource-efficient compilation of distributed quantum circuits for solving large-scale wireless communication network problems," arXiv:2501.10242, 2025.

[38] L. Lamport, "Time, clocks, and the ordering of events in a distributed system," *Communications of the ACM*, vol. 21, no. 7, pp. 558–565, 1978.

[39] ArduPilot Development Team, "ArduPilot: Open-source autopilot software," https://ardupilot.org, 2026.

[40] A. Koubâa, A. Allouch, M. Alajlan, Y. Javed, A. Belghith, and M. Khalgui, "Micro air vehicle link (MAVLink) in a nutshell: A survey," *IEEE Access*, vol. 7, pp. 87 658–87 680, 2019.

[41] H. W. Kuhn, "The Hungarian method for the assignment problem," *Naval Research Logistics Quarterly*, vol. 2, no. 1–2, pp. 83–97, 1955.

[42] V. Agafonkin *et al.*, "Leaflet: An open-source JavaScript library for interactive maps," https://leafletjs.com, 2026.

APPENDIX A
DESIGN NOTEBOOK

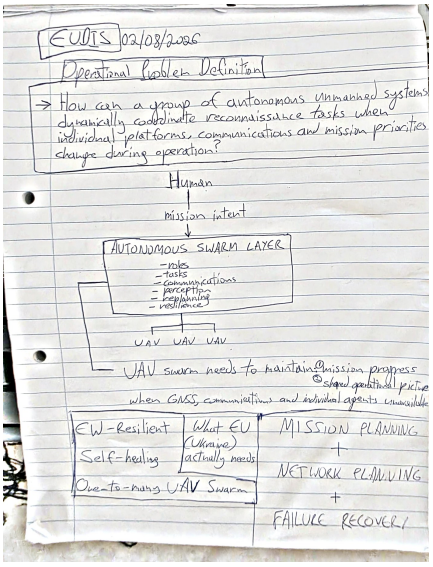
Fig. 9 reproduces the team's original design notebook pages (2 and 10 August 2026), from which Figs. 1 and 2, the network-loss states of Section III, the failure and safety behaviour of Section IV-E and the utility of Eq. (4) were developed. The photographs are cropped, with illumination flattened for print and content unaltered.

APPENDIX B
REPRODUCIBILITY

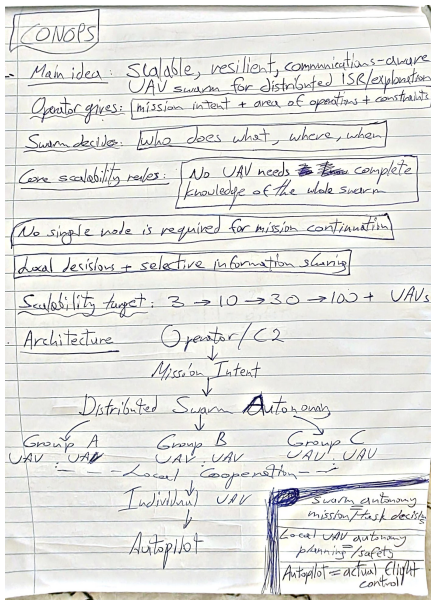
Every simulated number in Section VI is regenerated by the following commands of the `eudis-swarm` package; identical seeds give byte-identical CSV files. The range and lease sweeps are run once per control mode (option `--task-control`). The policy sweep is run once without a contract and once per policy (option `--degradation-policy`).

```
# range sweep (Fig. 6), once per mode
eudis-swarm-sweep --param communication-range \
  --from 30 --to 130 --steps 11 --seeds 20 \
  --jammer 40,50,15,3,8
# lease sweep (Fig. 7a), once per mode
eudis-swarm-sweep --param lease-timeout \
  --from 0.5 --to 6.0 --steps 12 --seeds 20 \
  --communication-range 50 \
  --failure-time 1000 --jammer 40,50,15,3,8
# degradation policy (Fig. 7b,c)
eudis-swarm-sweep --param communication-range \
  --from 30 --to 90 --steps 7 --seeds 20 \
  --failure-time 1000 --jammer 40,50,15,3,8
# detection floor without jammer (Sec. VI-F)
eudis-swarm-sweep --param communication-range \
  --from 50 --to 130 --steps 9 --seeds 20
# quorum floor (Table IV), split brain (Fig. 5)
eudis-swarm --scenario demo-quorum-floor
eudis-swarm --scenario baseline-quorum-floor
python -m eudis_swarm.task_claim_demo
# fleet/network and layout/fault (Tables V, VI)
eudis-swarm-generalization --seeds 20 --seed-base 1
eudis-swarm-robustness --seeds 20 --seed-base 1
# QAOA benchmark (Table VII), needs [quantum]
python paper/scripts/qaoa_benchmark.py
```

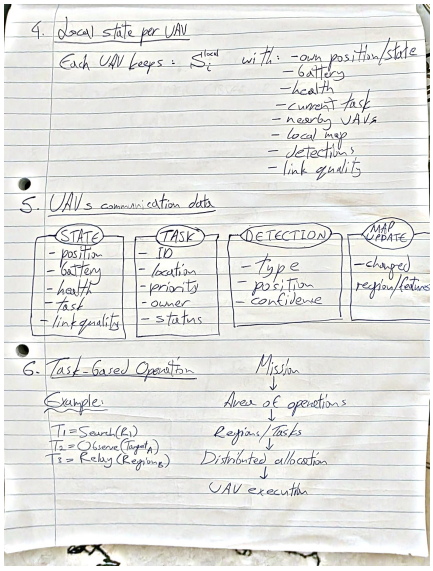
The committed raw CSV files of the fleet and layout studies have SHA-256 digests `f23f50f8...86e1fe` and `582018f1...f75576`. The source code is available from the author on request.



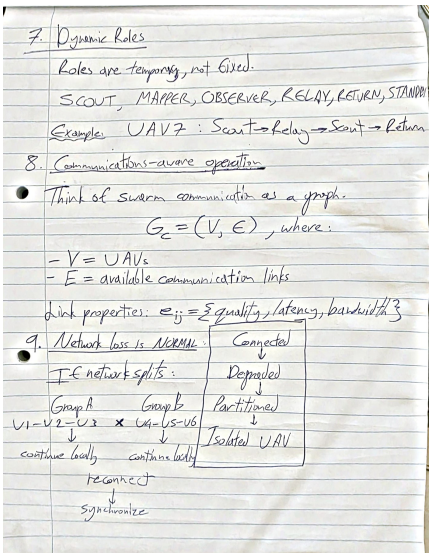
(a) Operational problem definition and the autonomous swarm layer.



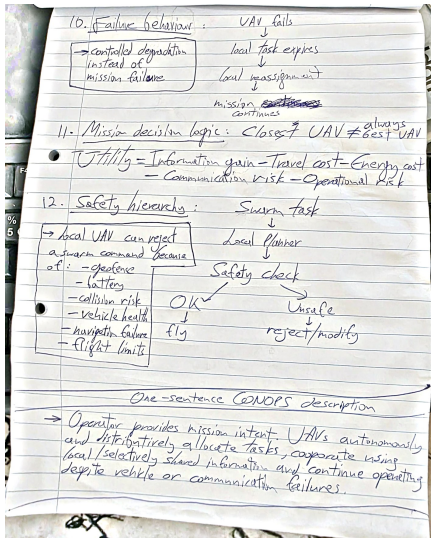
(b) CONOPS: main idea, scalability rules, architecture.



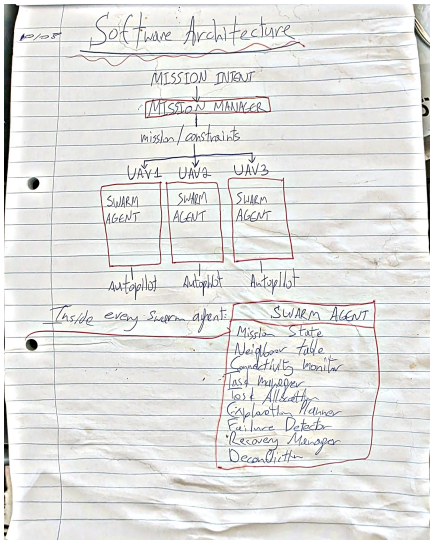
(c) Local state per UAV, message types, task-based operation.



(d) Dynamic roles; communication graph; network loss is normal.



(e) Failure behaviour, utility, safety hierarchy, one-sentence CONOPS.



(f) Software architecture: inside every swarm agent.

Fig. 9. Original design notebook pages of Team Teleforce.